



भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Indian Institute of Technology Hyderabad

IPACO Project Report CS5863

Optimizing CUDA Code to Maximize GPU Resource Overlap and
Improve Throughput

Team Members

CS24MTECH12018	Deeba Afridi
CS25MTECH02001	Sri Harika
CS24MTECH12017	Shivendra Deshpande

Contents

1	Problem Statement	2
2	Different Optimization Techniques and there Performance Analysis	2
	2.1 Resource Counter	2
	2.2 Warp Specialization for Mixed Precision	4
	2.3 Resource Counter Array	5
	2.4 CUDA Streams for Data Parallelism	6
3	Selection of Optimal Approach	6
4	GPU Architecture used for Performance Analysis	7
5	Performance analysis using Various kernels	7
	5.1 GEMM	7
	5.2 2MM	8
	5.3 3MM	8
6	Average & Speedup	9
	6.1 GEMM	9
	6.2 2MM	10
	6.3 3MM	11
7	NVIDIA Nsight Compute	12
8	Results	14

1 Problem Statement

Modern GPUs, such as those based on the Ampere architecture, utilize partitioned Streaming Multiprocessors (SMs) composed of multiple Streaming Multiprocessor Partitions (SMSPs), each with dedicated compute resources like FP32, INT32, FP64, and Tensor cores.

While this design improves energy efficiency, it often leads to resource underutilization due to homogeneous workloads and SIMT execution constraints that activate only one of available compute units. This project aims to address the inefficiency by identifying and applying static optimization techniques that enhance GPU resource utilization, specifically in matrix multiplication benchmarks from the PolyBench suite, through overlapping execution across diverse compute units, thereby improving overall performance without altering hardware architecture.



Figure 1: SM Architecture

2 Different Optimization Techniques and there Performance Analysis

We have identified and evaluated several static optimization techniques aimed at improving GPU resource utilization through compute resource overlap. To validate our approach, we used well-known matrix multiplication benchmarks from the PolyBench, including GEMM, 2MM, and 3MM. The following subsections discuss each of these static optimization techniques along with their observed performance impacts.

2.1 Resource Counter

Approach

- **Shared Resource Counters**

Two counter variables, `fp32Count` and `fp64Count`, declared as `__shared__` and initialized to 0 are allocated in the shared memory, with each counter tracking the number of FP32 and FP64 compute units that are occupied by the warps in a SM.

- **Thread Zero of Every Warp Determines Compute Type**

Only one thread per warp (typically thread 0) first attempts to acquire FP32 resource unit using `fp32Count` counter variable by seeing it's current value.

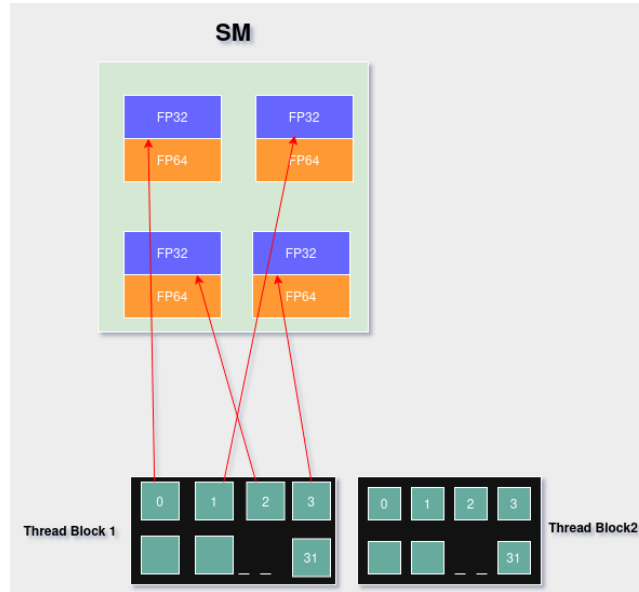


Figure 2: Initial warps accessing FP32 resource units

If FP32 slots are exhausted, it tries to acquire FP64 using `fp64Count` resource counter depending on the value of resource counter. Maximum available resources of both the types in a SM is equal to 4.

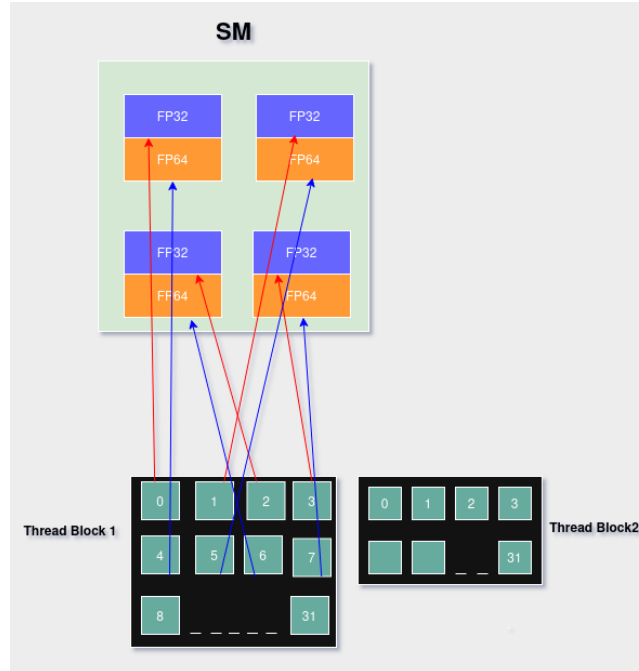


Figure 3: Warps accessing FP64 when all FP32 resource units are busy

Once the compute type is determined, the result is broadcast to all threads in the warp, ensuring consistent execution across the warp.

If all the resource units are busy with the computation task at a certain point of time then warps will stall and wait for the resource units to get freed.

- **Computation Phase**

Each thread computes one element of the output matrix. If the warp is assigned FP32 resources, computations are performed using `float` types. Otherwise, input data is typecast to `double`, and FP64 units are used for computation.

- **Releasing Resources**

After computation, the warp leader releases the resource slot by performing an on the appropriate counter. This allows subsequent warps to acquire the now-freed compute unit slot.

Performance Analysis

The above Counter approach aims to improve utilization by overlapping FP32 and FP64 resource usage within an SM. With this approach we were able to achieve the goal and also beat the baseline kernels mainly because of the overlap of resource units.

Host-Side Typecasting of Input Data

An additional optimization was attempted by sending double-precision data from the host to the device, instead of typecasting on the GPU. However, this change did not yield any significant improvement in performance, likely because memory access latency remains the dominant overhead. But still the average execution time of the optimized kernel was lesser as compared to the average execution time of baseline kernels.

2.2 Warp Specialization for Mixed Precision

- **Techniques Employed:**

- **Fixed Block Sizing:** Utilizes a predetermined block configuration (e.g., `BLOCK.SIZE.X = 16`, `BLOCK.SIZE.Y = 16`) for thread organization.
- **Warp-Level Computation Path Specialization (Static):** Warps within a thread block are statically assigned to either FP64 or FP32 computation paths based on `warpId < FP64.WARPS`.
- **Direct Global Memory Writes for Results:** Each thread writes its computed result directly to global memory without intermediate shared memory buffering.
- **Explicit Type Casting:** Inputs and scalar parameters are cast to the appropriate precision (float or double) at the start of each execution path, and results are cast back to `DATA.TYPE` before being stored.

- **Approach:**

- **Thread Identification and Work Assignment:** Each thread computes its global row and column indices to determine its target output element.
- **Warp Path Determination:** `warpId` is calculated using thread indices. Warps with ID less than `FP64.WARPS` take the FP64 path, others take the FP32 path.
- **Precision-Specific Computation:**
 - * **FP64 Path:** Accumulator and operands are cast to `double`. All arithmetic is performed in double precision.

- * **FP32 Path:** Accumulator and operands are cast to `float`. All arithmetic is performed in single precision.
- **Result Storage:** The final computed value is cast to `DATA_TYPE` and written directly to the appropriate position in the global output matrix.
- **Synchronization for Final Results:** No `__syncthreads()` is required since each thread writes to a unique location in global memory.
- **Performance Analysis:**
 - **Global Memory Traffic:** Efficient coalesced writes are expected since threads access contiguous output elements. However, initial reads (e.g., for `D = beta * D + ...`) still contribute to memory load.
 - **Static Warp Assignment Limitations:** The fixed ratio of FP64 to FP32 warps may not align with runtime execution unit availability or problem-specific precision needs, potentially leading to underutilization.
 - **Mixed-Precision Logic Overhead:** Casting between `float`, `double`, and `DATA_TYPE` adds instruction overhead. Gains from using FP32 must outweigh these costs to yield performance benefits.

2.3 Resource Counter Array

Approach

- **Global Device-Side Resource Counters**

Two arrays, `fp32Count` and `fp64Count`, are allocated in global device memory, with each array having size equal to the number of Streaming Multiprocessors (SMs). Each index in the array corresponds to a specific SM.

These counters track how many warps on each SM are currently using FP32 and FP64 compute units, respectively.

- **Per-SM Resource Management Using `smId`**

Each thread retrieves its SM ID. This `smId` is used to index into the resource counter arrays, ensuring correct tracking for the corresponding SM.

- **Warp Leader Determines Compute Type**

Only one thread per warp (typically thread 0) attempts to acquire a slot in the `fp32Count` array. If FP32 slots are exhausted, it tries to acquire a slot in the `fp64Count` array instead.

Once the compute type is determined, the result is broadcast to all threads in the warp, ensuring consistent execution across the warp.

- **Computation Phase**

Each thread computes one element of the output matrix. If the warp is assigned FP32 resources, computations are performed using `float` types. Otherwise, input data is typecast to `double`, and FP64 units are used for computation.

- **Releasing Resources**

After computation, the warp leader releases the resource slot by performing an on the appropriate counter. This allows subsequent warps to acquire the now-freed compute unit slot.

Performance Analysis

Although the Resource Counter Array approach aims to improve utilization by overlapping FP32 and FP64 resource usage within an SM, its performance does not surpass the baseline GEMM kernel. The primary bottleneck arises from the use of global memory to maintain the resource counters. Accessing global memory introduces latency, which offsets any gains from improved compute resource balancing.

Host-Side Typecasting of Input Data

An additional optimization was attempted by sending double-precision data from the host to the device, instead of typecasting on the GPU. However, this change did not yield any significant improvement in performance, likely because memory access latency remains the dominant overhead.

2.4 CUDA Streams for Data Parallelism

- **Techniques Employed:**

- CUDA Streams for Task Parallelism.
- Kernel modified to support row offsets.

- **Approach:**

- Output matrix **C** is divided into two halves.
- Each half is processed in a different CUDA stream using the same kernel.
- Both kernels read full matrices **A** and **B**; compute only partial **C**.

- **Performance Analysis:**

- Gains may come from improved occupancy and SM utilization.
- **A** and **B** duplication in memory access may strain cache.
- Simpler than tiling, but outperformed by tiled approaches.

3 Selection of Optimal Approach

Among the approaches discussed above, the **Resource Counter** method and the **warp specialization** method has demonstrated performance comparable to the baselines across various Polybench kernels. Because of its consistent performance, we have selected the above two approaches for further analysis and performance evaluation in the subsequent sections.

4 GPU Architecture used for Performance Analysis

- DRAM size: 80 GB
- Architecture: Ampere
- Model: NVIDIA A100
- Driver version: 525.60.13
- CUDA version: 12.0

5 Performance analysis using Various kernels

Performance of our approach is tested using various polybench kernels like GEMM, 2MM and 3MM. The performance with respect to each kernel is detailed in subsequent sections.

5.1 GEMM

We have implemented the above discussed Resource Counter approach for GEMM Kernel and compared the performance with default GEMM kernel for dataset sizes 512, 1024, 2048, 4096.

The comparison results are displayed in Table 1 below :

Run-Type	GPU Used	Dataset Size	Run-1	Run-2	Run-3	Run-4	Run-5
Baseline	A100-80GB	512	0.346	0.346	0.328	0.341	0.341
Resource counter	A100-80GB	512	0.142	0.142	0.143	0.139	0.142
Warp specialization	A100-80GB	512	0.145	0.146	0.145	0.147	0.144
Baseline	A100-80GB	1024	1.843	1.843	1.853	1.840	1.840
Resource counter	A100-80GB	1024	0.844	0.826	0.832	0.828	0.828
Warp specialization	A100-80GB	1024	0.823	0.826	0.826	0.826	0.826
Baseline	A100-80GB	2048	14.028	14.014	14.015	14.010	13.999
Resource counter	A100-80GB	2048	6.285	6.290	6.322	6.298	6.317
Warp specialization	A100-80GB	2048	6.274	6.255	6.274	6.277	6.269
Baseline	A100-80GB	4096	151.195	150.940	151.247	151.150	151.182
Resource counter	A100-80GB	4096	58.375	58.349	58.077	58.378	58.417
Warp specialization	A100-80GB	4096	55.647	55.699	55.542	55.650	55.655

Table 1: GEMM Execution Times on A100-80GB GPU (in milliseconds)

5.2 2MM

We have implemented the above discussed Resource Counter approach for 2MM Kernel and compared the performance with default GEMM kernel for dataset sizes 512, 1024, 2048, 4096.

The comparison results are displayed in Table 2 below :

Run-Type	GPU Used	Dataset Size	Run-1	Run-2	Run-3	Run-4	Run-5
Baseline	A100-80GB	512	0.668	0.650	0.667	0.660	0.660
Resource counter	A100-80GB	512	0.264	0.268	0.274	0.271	0.270
Warp specialization	A100-80GB	512	0.259	0.258	0.260	0.259	0.259
Baseline	A100-80GB	1024	3.704	3.704	3.681	3.679	3.711
Resource counter	A100-80GB	1024	1.649	1.643	1.657	1.655	1.644
Warp specialization	A100-80GB	1024	1.593	1.591	1.593	1.602	1.603
Baseline	A100-80GB	2048	28.132	28.049	28.157	28.057	28.148
Resource counter	A100-80GB	2048	12.451	12.380	12.477	12.432	12.488
Warp specialization	A100-80GB	2048	12.283	12.223	12.284	12.278	12.289
Baseline	A100-80GB	4096	299.658	299.158	299.572	299.799	299.671
Resource counter	A100-80GB	4096	127.010	126.830	126.675	126.723	126.619
Warp specialization	A100-80GB	4096	118.147	118.419	118.368	118.323	118.473

Table 2: 2MM Execution Times on A100-80GB GPU (in milliseconds)

5.3 3MM

We have implemented the above discussed Resource counter approach for 3MM Kernel and compared the performance with default 3MM kernel for dataset sizes 512, 1024, 2048, 4096.

The comparison results are displayed in Table 3 below :

Run-Type	GPU Used	Dataset Size	Run-1	Run-2	Run-3	Run-4	Run-5
Baseline	A100-80GB	512	0.986	0.978	0.979	0.975	0.979
Resource counter	A100-80GB	512	0.377	0.378	0.320	0.317	0.318
Warp specialization	A100-80GB	512	0.376	0.373	0.376	0.374	0.378
Baseline	A100-80GB	1024	5.560	5.498	5.517	5.540	5.517
Resource counter	A100-80GB	1024	1.987	1.991	1.983	1.995	1.985
Warp specialization	A100-80GB	1024	2.393	2.380	2.384	2.371	2.384
Baseline	A100-80GB	2048	42.018	42.042	42.092	42.032	42.124
Resource counter	A100-80GB	2048	14.940	14.959	14.923	14.968	14.962
Warp specialization	A100-80GB	2048	18.301	18.406	18.399	18.400	18.396
Baseline	A100-80GB	4096	444.894	443.742	444.305	444.198	442.883
Resource counter	A100-80GB	4096	198.600	198.258	198.474	197.824	198.098
Warp specialization	A100-80GB	4096	180.451	180.367	180.197	180.309	180.498

Table 3: 3MM Execution Times on A100-80GB GPU (in milliseconds)

6 Average & Speedup

6.1 GEMM

Average runs of 5 runs of GEMM kernel and speedup over baseline :

RUN-TYPE	GPU Used	Kernel	Dataset Size	Average	Speedup
Baseline	A100-80GB	GEMM	512	0.3404	1.000
Resource counter	A100-80GB	GEMM	512	0.1416	2.404
Warp specialization	A100-80GB	GEMM	512	0.1454	2.341
Baseline	A100-80GB	GEMM	1024	1.8438	1.000
Resource counter	A100-80GB	GEMM	1024	0.8316	2.217
Warp specialization	A100-80GB	GEMM	1024	0.8246	2.236
Baseline	A100-80GB	GEMM	2048	14.0312	1.000
Resource counter	A100-80GB	GEMM	2048	6.3024	2.226
Warp specialization	A100-80GB	GEMM	2048	6.2698	2.238
Baseline	A100-80GB	GEMM	4096	151.1428	1.000
Resource counter	A100-80GB	GEMM	4096	58.3192	2.592
Warp specialization	A100-80GB	GEMM	4096	55.6386	2.717

Table 4: Performance of GEMM Kernel on A100-80GB GPU for Various Dataset Sizes and Optimization Techniques

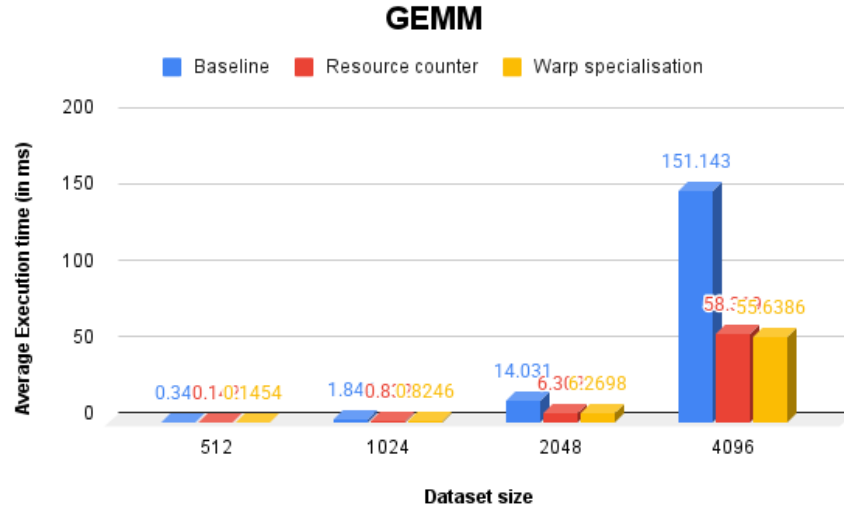


Figure 4: Average Execution time(in ms) across different datasets for Baseline, Resource counter and Warp specialization

6.2 2MM

Average runs of 5 runs of 2MM kernel and speedup over baseline :

RUN-TYPE	GPU Used	Kernel	Dataset Size	Average	Speedup
Baseline	A100-80GB	GEMM	512	0.6610	1.000
Resource counter	A100-80GB	GEMM	512	0.2694	2.454
Warp specialization	A100-80GB	GEMM	512	0.2590	2.552
Baseline	A100-80GB	GEMM	1024	3.6958	1.000
Resource counter	A100-80GB	GEMM	1024	1.6496	2.240
Warp specialization	A100-80GB	GEMM	1024	1.5964	2.315
Baseline	A100-80GB	GEMM	2048	28.1020	1.000
Resource counter	A100-80GB	GEMM	2048	12.4456	2.258
Warp specialization	A100-80GB	GEMM	2048	12.2714	2.290
Baseline	A100-80GB	GEMM	4096	299.5716	1.000
Resource counter	A100-80GB	GEMM	4096	126.7714	2.363
Warp specialization	A100-80GB	GEMM	4096	118.3460	2.531

Table 5: Performance of 2MM Kernel on A100-80GB GPU for Various Dataset Sizes and Optimization Techniques

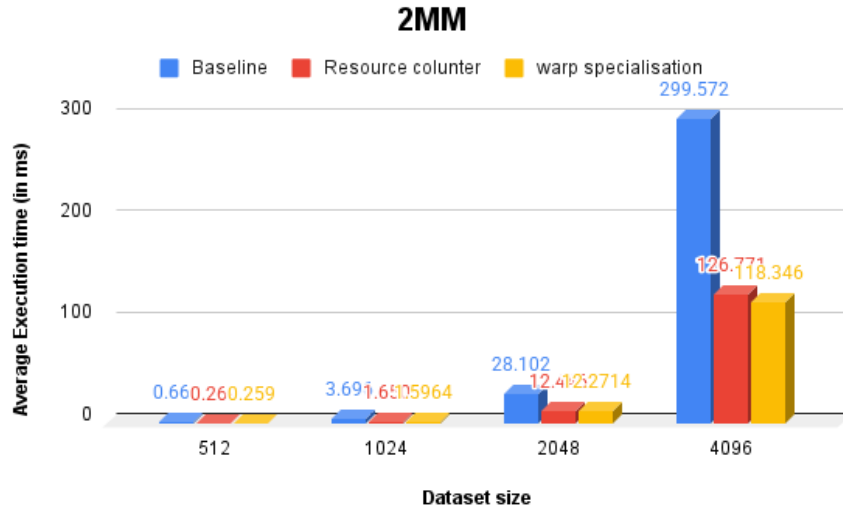


Figure 5: Average Execution time(in ms) across different datasets for Baseline, Resource counter and Warp specialization

6.3 3MM

Average runs of 5 runs of 3MM kernel and speedup over baseline :

RUN-TYPE	GPU Used	Kernel	Dataset Size	Average	Speedup
Baseline	A100-80GB	GEMM	512	0.97940	1.000
Resource counter	A100-80GB	GEMM	512	0.34200	2.864
Warp specialization	A100-80GB	GEMM	512	0.37540	2.609
Baseline	A100-80GB	GEMM	1024	5.52640	1.000
Resource counter	A100-80GB	GEMM	1024	1.98920	2.778
Warp specialization	A100-80GB	GEMM	1024	2.38240	2.320
Baseline	A100-80GB	GEMM	2048	42.07960	1.000
Resource counter	A100-80GB	GEMM	2048	14.95040	2.815
Warp specialization	A100-80GB	GEMM	2048	18.38040	2.289
Baseline	A100-80GB	GEMM	4096	444.00440	1.000
Resource counter	A100-80GB	GEMM	4096	198.25080	2.240
Warp specialization	A100-80GB	GEMM	4096	180.36440	2.462

Table 6: Average Execution times and speedups for 3MM kernel on A100-80GB GPU with varying dataset sizes and optimization strategies.

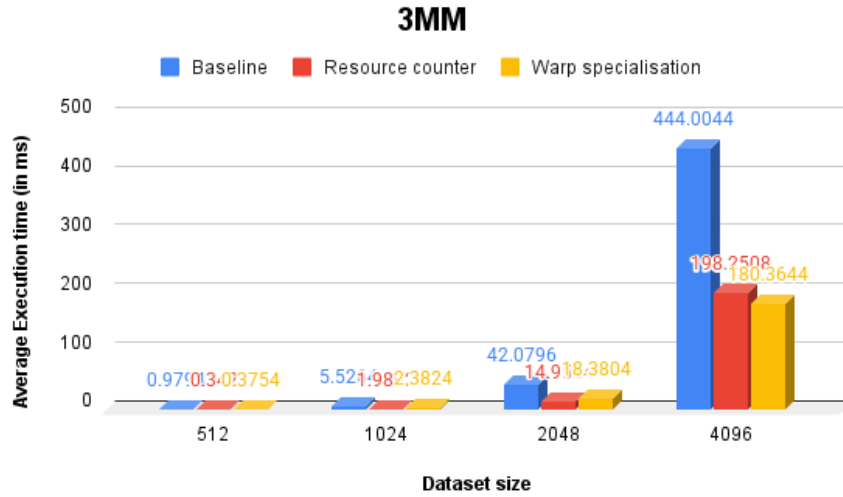


Figure 6: Average Execution time(in ms) across different datasets for Baseline, Resource counter and Warp specialization

7 NVIDIA Nsight Compute

GPU Throughput

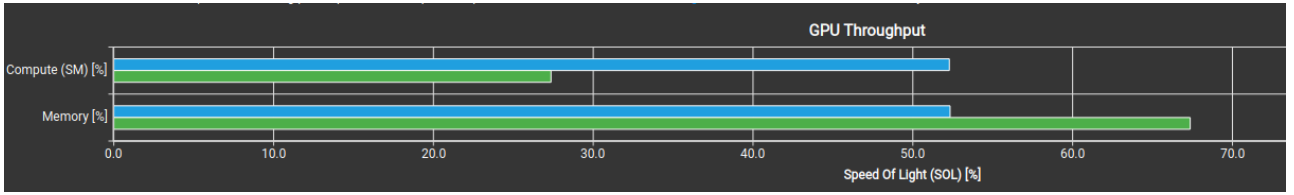


Figure 7: GPU Throughput between Baseline and Resource Counter with dataset 4096

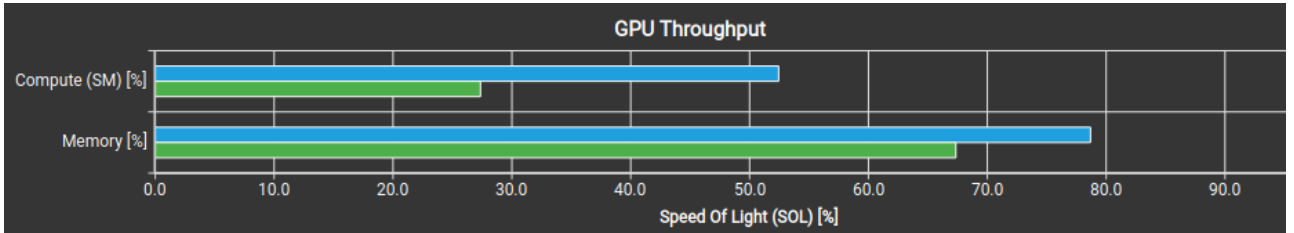


Figure 8: GPU Throughput between Baseline and Warp Specialization with dataset 4096

Pipe Utilization

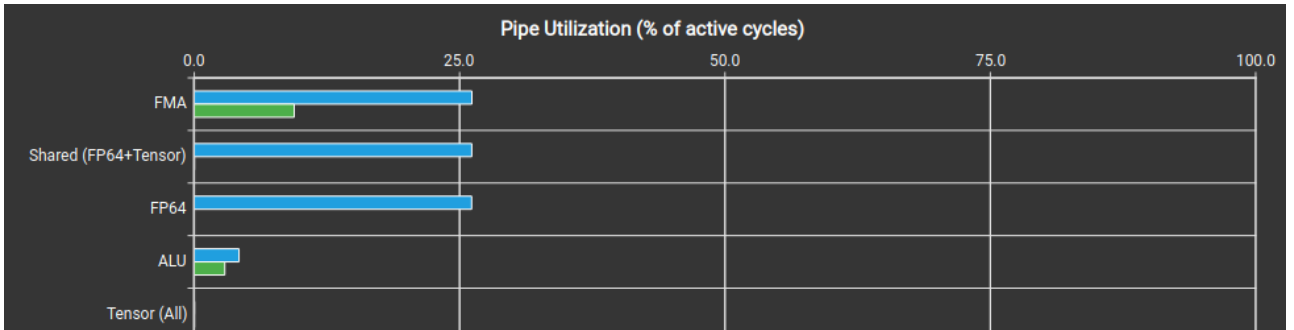


Figure 9: Pipe Utilization (% of active cycles) between Baseline and Resource Counter with dataset 4096

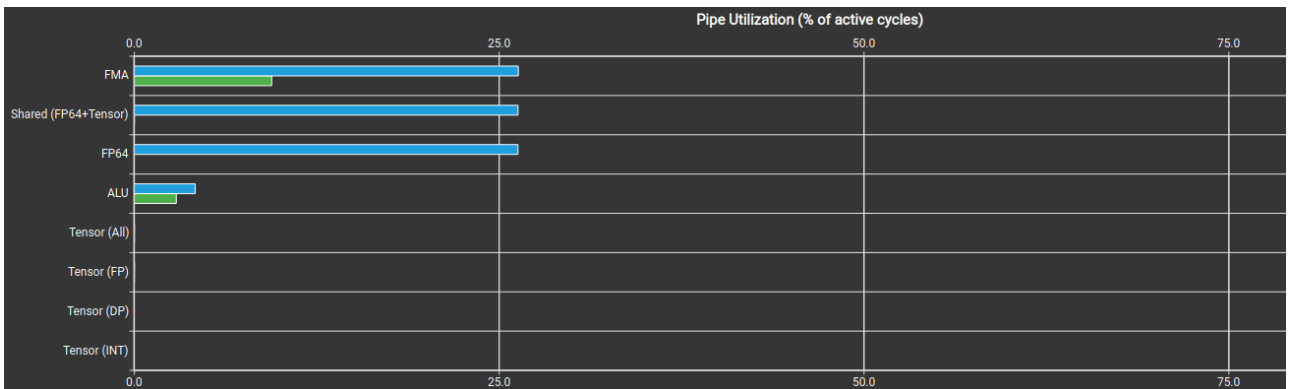


Figure 10: Pipe Utilization (% of active cycles) between Baseline and Warp specialization with dataset 4096

Executed Instruction Mix



Figure 11: Executed Instruction Mix between Baseline and Resource Counter with dataset 4096

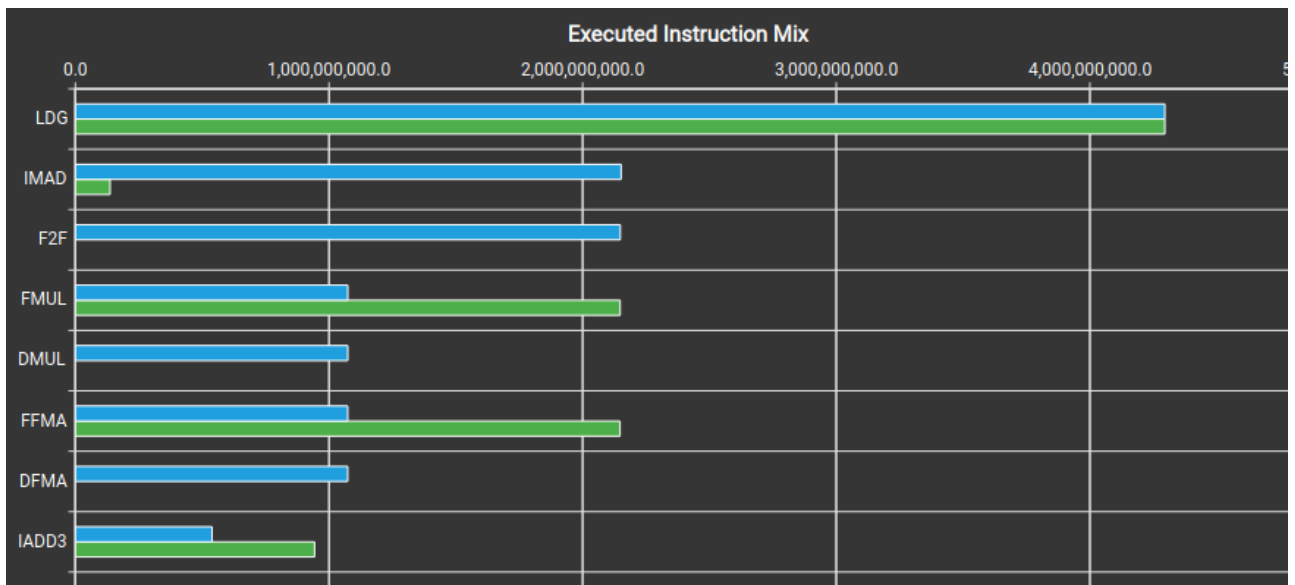


Figure 12: Executed Instruction Mix between Baseline and Warp specialization with dataset 4096

Warps Per Scheduler

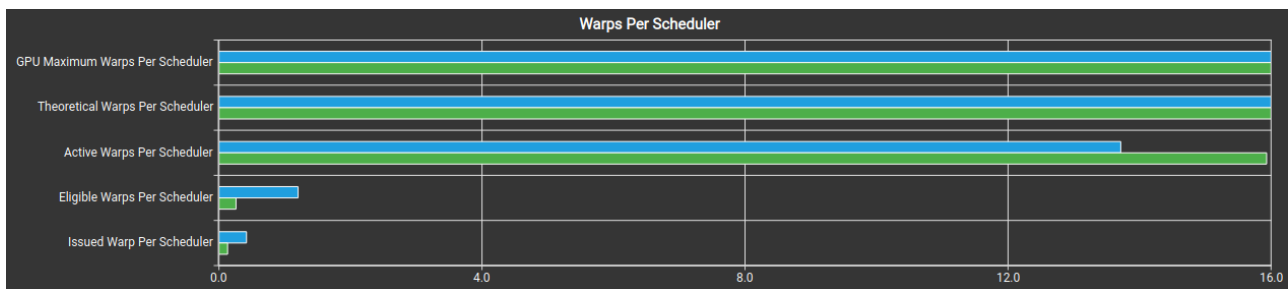


Figure 13: Warps Per Scheduler between Baseline and Resource Counter with dataset 4096

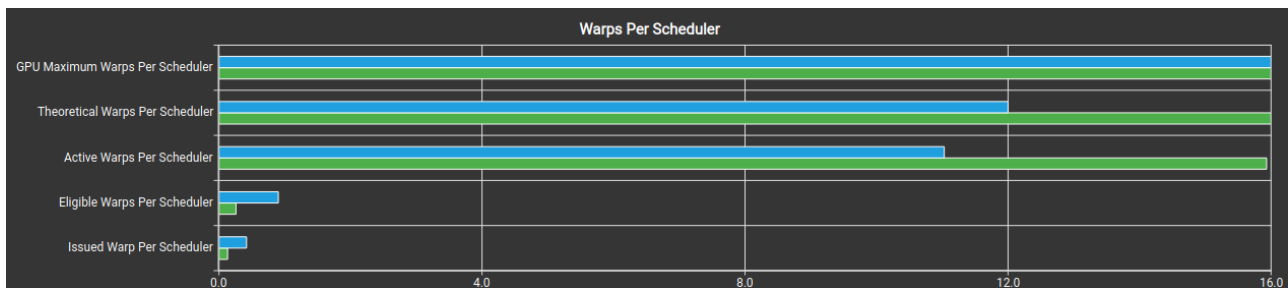


Figure 14: Warps Per Scheduler between Baseline and Warp specialization with dataset 4096

8 Results

Code : [Github link](#)

Runs : Results for Baseline and proposed optimizations